

CPU
Scheduling

CPU Scheduling

Preemptive

Non-
Preemptive

Mata Kuliah Sistem Operasi

Shortest
Remaining
Job First

Longest
Remaining
Job First

Round
Robin

First Come
First Serve

Shortest
Job First

Longest Jo
First

Program Studi Teknik Informatika

Universitas Muhammadiyah Jember

Tahun Akademik 2025

Dosen Pengampu: Triawan Adi Cahyanto, M.Kom.

Daftar Isi

1 Basic Concepts



2 Scheduling Criteria



3 Scheduling Algorithms



4 Thread Scheduling



5 Multi-Processor
Scheduling



6 Real-Time CPU Scheduling



7 Operating-System
Examples



8 Algorithm Evaluation



9 Summary



Basic Concepts

🕒 Apa itu CPU Scheduling?

Proses yang digunakan oleh sistem operasi untuk **memutuskan tugas atau proses mana** yang akan menggunakan CPU pada waktu tertentu. CPU hanya dapat menangani satu tugas pada satu waktu, namun biasanya ada banyak tugas yang perlu diproses.

➡ Arrival Time

Waktu ketika proses tiba di antrian siap (ready queue)

✅ Completion Time

Waktu ketika proses selesai dieksekusi

🕒 Burst Time

Waktu yang dibutuhkan proses untuk eksekusi CPU

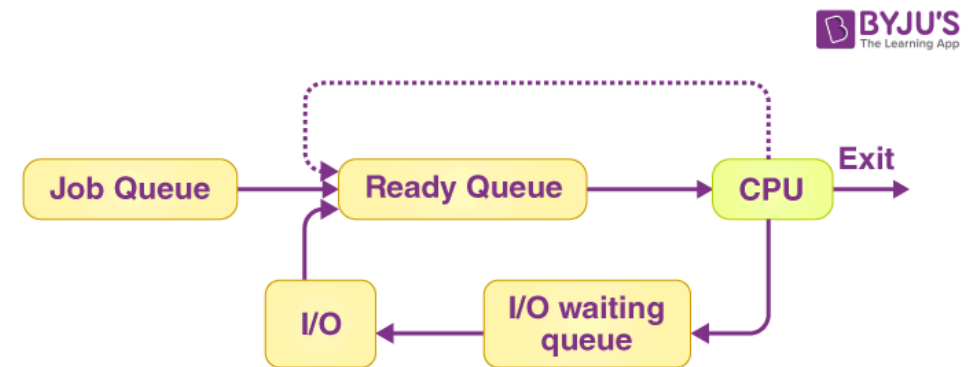
🔄 Turn Around Time

Waktu dari kedatangan hingga penyelesaian proses

⌚ Waiting Time

Waktu yang dihabiskan proses menunggu di antrian siap

Diagram CPU Scheduling



Scheduling Criteria

Kriteria penjadwalan CPU digunakan untuk **mengevaluasi performa algoritma penjadwalan**. Setiap kriteria memiliki fokus yang berbeda dalam mengukur efisiensi sistem.



CPU Utilization

Persentase waktu CPU yang digunakan. Tujuan: **maksimalkan** penggunaan CPU agar tidak menganggur.



Throughput

Jumlah proses yang selesai per satuan waktu. Tujuan: **maksimalkan** jumlah proses yang diselesaikan.



Turnaround Time

Waktu dari kedatangan hingga penyelesaian proses. Tujuan: **minimalkan** waktu total eksekusi.



Waiting Time

Waktu proses menunggu di antrian siap. Tujuan: **minimalkan** waktu tunggu untuk respons lebih cepat.



Response Time

Waktu dari permintaan hingga respons pertama. Penting untuk sistem interaktif agar **responsif**.



Completion Time

Waktu ketika proses selesai dieksekusi. Digunakan untuk menghitung metrik performa lainnya.



Priority

Tingkat kepentingan proses. Proses dengan prioritas **lebih tinggi** dieksekusi terlebih dahulu.



Predictability

Konsistensi waktu eksekusi proses. Proses harus berjalan dengan waktu yang **konsisten** dalam kondisi serupa.

Scheduling Algorithms

Algoritma penjadwalan CPU menentukan **urutan proses** yang akan dieksekusi. Setiap algoritma memiliki karakteristik dan kecocokan untuk situasi yang berbeda.

Non-Preemptive Scheduling

Proses yang sedang berjalan **tidak dapat diinterupsi** hingga selesai atau menunggu I/O.

1 FCFS (First Come First Served)

2 SJF (Shortest Job First)

3 LJF (Longest Job First)

Preemptive Scheduling

Proses yang sedang berjalan **dapat diinterupsi** oleh proses lain dengan prioritas lebih tinggi.

1 Priority Scheduling

2 Round Robin

3 SRTF (Shortest Remaining Time First)

4 LRTF (Longest Remaining Time First)

5 Multilevel Queue Scheduling

6 Multilevel Feedback Queue

↔ Perbedaan Utama

Non-Preemptive

- ✓ Sederhana, mudah diimplementasikan
- ✓ Overhead konteks switching rendah
- ✗ Responsivitas rendah untuk proses interaktif

Preemptive

- ✓ Responsivitas tinggi untuk proses interaktif
- ✓ Mencegah proses lama mendominasi CPU
- ✗ Overhead konteks switching lebih tinggi

Scheduling Algorithms Details

Mari kita pelajari lebih detail tentang beberapa algoritma penjadwalan CPU yang **paling umum** digunakan dalam sistem operasi.

1 FCFS

i Cara Kerja

Proses yang **datang pertama** akan dilayani pertama. Mirip dengan antrian di kasir supermarket.

Contoh:

Proses	Waktu Kedatangan	Waktu Burst
P1	0	24
P2	1	3
P3	2	3

P1 P2 P3 0 24 27 30

👍 Kelebihan

- ✓ Sederhana & mudah diimplementasikan
- ✓ Tidak ada starvation

👎 Kekurangan

- ✗ Waktu tunggu rata-rata tinggi
- ✗ Efek convoy (proses pendek tertahan)

2 SJF

i Cara Kerja

Proses dengan **waktu burst terpendek** akan dipilih untuk dieksekusi berikutnya.

Contoh:

Proses	Waktu Kedatangan	Waktu Burst
P1	0	6
P2	2	8
P3	4	7
P4	5	3

P1 P4 P3 P2 0 6 9 16 24

👍 Kelebihan

- ✓ Waktu tunggu rata-rata minimum
- ✓ Throughput tinggi

👎 Kekurangan

- ✗ Sulit memprediksi waktu burst
- ✗ Dapat menyebabkan starvation

3 Priority Scheduling

i Cara Kerja

Setiap proses diberi **prioritas**. Proses dengan prioritas tertinggi akan dieksekusi pertama.

Contoh:

Proses	Waktu Burst	Prioritas
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

P2 P5 P1 P4 0 1 6 16 18 19

👍 Kelebihan

- ✓ Fleksibel untuk berbagai kebutuhan
- ✓ Dapat digunakan untuk sistem real-time

👎 Kekurangan

- ✗ Dapat menyebabkan starvation
- ✗ Solusi: aging (menaikkan prioritas lama)

Scheduling Algorithms Details (Part 2)

Mari kita lanjutkan pembahasan tentang algoritma penjadwalan CPU lainnya yang **sama pentingnya** dalam sistem operasi modern.



Round Robin

Cara Kerja

Setiap proses mendapat **waktu quantum** yang sama. Jika proses belum selesai, akan kembali ke antrian.

Contoh (Quantum = 4):

Proses	Waktu Burst
P1	24
P2	3
P3	3

P1 P2 P3 P1 P1 P1 P1 0 4 7 10 14 18 22 26 30

Quantum biasanya 10-100ms

Kelebihan

- ✓ Responsif untuk sistem interaktif
- ✓ Tidak ada starvation

Kekurangan

- × Waktu tunggu rata-rata lebih tinggi
- × Overhead konteks switching tinggi



SRTF

Cara Kerja

Versi **preemptive** dari SJF. Proses dengan waktu tersisa terpendek akan dieksekusi.

Contoh:

Proses	Waktu Kedatangan	Waktu Burst
P1	0	8
P2	1	4
P3	2	9
P4	3	5

P2 P4 P1 P3 0 1 5 10 13 26

Kelebihan

- ✓ Waktu tunggu rata-rata minimum
- ✓ Responsif untuk proses pendek

Kekurangan

- × Sulit memprediksi waktu burst
- × Overhead konteks switching tinggi



Multilevel Queue

Cara Kerja

Antrian siap dibagi menjadi **beberapa antrian** dengan prioritas berbeda. Setiap antrian memiliki algoritma sendiri.

Struktur Antrian:

Foreground:

Round Robin

Proses interaktif

Background:

FCFS

Proses batch

Scheduling Antar Antrian

- Prioritas tetap: Proses foreground diutamakan
- Time slice: 80% CPU untuk foreground, 20% untuk background

Kelebihan

- ✓ Fleksibel untuk berbagai jenis proses
- ✓ Dapat mengoptimalkan berbagai kriteria

Kekurangan

- × Proses tidak dapat berpindah antrian
- × Dapat menyebabkan starvation

Contoh Praktis 1: FCFS

Mari kita lihat contoh perhitungan dengan algoritma **First Come First Served (FCFS)**, di mana proses yang datang pertama akan dilayani pertama.

Data Proses

Process ID	Arrival Time	Burst Time
P1	0	24
P2	1	3
P3	2	3

Perhitungan

- 1 Menentukan **Waiting Time** untuk setiap proses:

$$WT = \text{Start Time} - \text{Arrival Time}$$

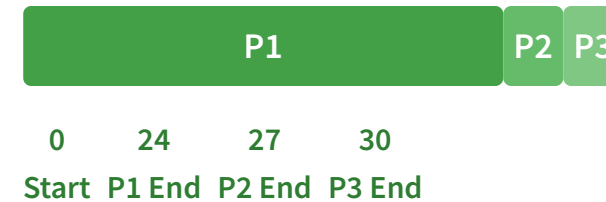
- 2 Menentukan **Turnaround Time** untuk setiap proses:

$$TAT = \text{Completion Time} - \text{Arrival Time}$$

- 3 Menghitung **Average Waiting Time**:

$$AWT = (WT_1 + WT_2 + WT_3) / 3$$

Gantt Chart FCFS



Σ Hasil Perhitungan

- 1 **Waiting Time:**

- P1: $0 - 0 = 0$
- P2: $24 - 1 = 23$
- P3: $27 - 2 = 25$

- 2 **Turnaround Time:**

- P1: $24 - 0 = 24$
- P2: $27 - 1 = 26$
- P3: $30 - 2 = 28$

$$\text{Average Waiting Time} = (0 + 23 + 25) / 3 = 16$$

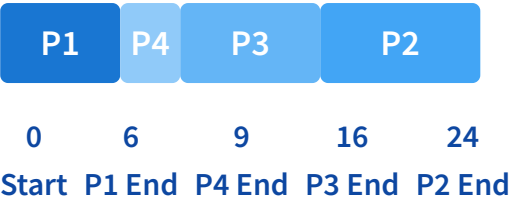
Contoh Praktis 2: SJF

Mari kita lihat contoh perhitungan dengan algoritma **Shortest Job First (SJF)**, di mana proses dengan waktu burst terpendek akan diprioritaskan.

Data Proses

Process ID	Arrival Time	Burst Time
P1	0	6
P2	2	8
P3	4	7
P4	5	3

Gantt Chart SJF



Perhitungan SJF

- Urutkan proses berdasarkan **waktu burst terpendek** saat CPU tersedia
- Hitung **Waiting Time**: $WT = \text{Start Time} - \text{Arrival Time}$
- Hitung **Turnaround Time**:
 $TAT = \text{Completion Time} - \text{Arrival Time}$

Hasil Perhitungan

- Waiting Time:**
 - P1: $0 - 0 = 0$
 - P2: $16 - 2 = 14$
 - P3: $9 - 4 = 5$
 - P4: $6 - 5 = 1$

$\text{Average Waiting Time} = (0 + 14 + 5 + 1) / 4 = 5$

Perbandingan SJF vs FCFS

Algoritma	Average Waiting Time	Average Turnaround Time	Keunggulan
FCFS	7.75	13.75	Sederhana, tidak ada starvation

Algoritma	Average Waiting Time	Average Turnaround Time	Keunggulan
SJF	5.0	11.5	Waktu tunggu minimum

Contoh Praktis 3: Round Robin

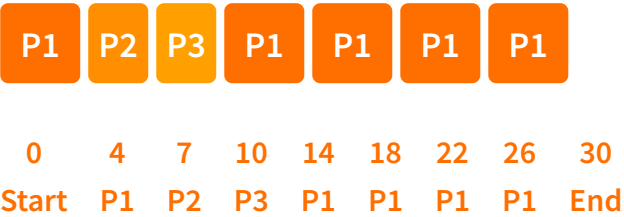
Mari kita lihat contoh perhitungan dengan algoritma **Round Robin**, di mana setiap proses mendapat jatah waktu (quantum) yang sama untuk dieksekusi.

Data Proses

Process ID	Arrival Time	Burst Time
P1	0	24
P2	1	3
P3	2	3

 Time Quantum = 4

Gantt Chart Round Robin



Perhitungan Round Robin

- 1 Setiap proses dieksekusi selama **time quantum** (4 unit waktu)
- 2 Jika proses belum selesai, kembali ke **akhir antrian**
- 3 Hitung **Waiting Time** dan **Turnaround Time**

Hasil Perhitungan

- 1 **Waiting Time:**
- P1: $(6+3+3+3+3) = 18$
 - P2: $(4-1) = 3$
 - P3: $(7-2) = 5$

Average Waiting Time = $(18 + 3 + 5) / 3 = 8.67$

Perbandingan Algoritma

Algoritma	Average Waiting Time	Average Turnaround Time	Keunggulan
FCFS	17.0	22.0	Sederhana, tidak ada starvation
SJF	5.0	11.5	Waktu tunggu minimum

Algoritma	Average Waiting Time	Average Turnaround Time	Keunggulan
Round Robin	8.67	15.67	Responsif, adil untuk semua proses

Thread Scheduling

Thread scheduling adalah proses penjadwalan thread untuk dieksekusi pada CPU. Ada dua tingkatan thread yang perlu dipahami: **User-Level Threads (ULT)** dan **Kernel-Level Threads (KLT)**.

User-Level Threads (ULT)

Thread yang dikelola oleh **thread library** di ruang pengguna, tidak terlihat oleh kernel.

- ✓ Dijadwalkan ke Lightweight Process (LWP)
- ✓ Konteks switching lebih cepat
- ✓ Dapat berjalan pada OS yang tidak support thread

Kernel-Level Threads (KLT)

Thread yang dikelola langsung oleh **kernel sistem operasi**.

- ✓ Setiap thread terlihat oleh kernel
- ✓ Dapat berjalan pada multiprocessor
- ✓ Blocking satu thread tidak memblokir proses

Contention Scope

Menentukan **tingkat kompetisi** thread untuk mendapatkan sumber daya CPU.

- 1 Process Contention Scope (PCS):** Kompetisi antar thread dalam satu proses
- 2 System Contention Scope (SCS):** Kompetisi antar semua thread dalam sistem

Allocation Domain

Sekumpulan **sumber daya** (CPU cores) yang dapat digunakan oleh thread untuk bersaing.

- ✓ Pada sistem multicore, ada beberapa domain alokasi
- ✓ Setiap domain terdiri dari satu atau lebih core
- ✓ Thread dapat menjadi bagian dari beberapa domain

Kelebihan PCS vs SCS

Process Contention Scope (PCS)

- Context switching di ruang pengguna (lebih cepat)
- Mengurangi pemanggilan sistem
- Performa lebih baik
- Lebih murah dalam implementasi

System Contention Scope (SCS)

- Setiap thread memerlukan LWP terpisah
- Meningkatkan kompleksitas kernel
- Lebih mahal dalam implementasi
- Sinkronisasi lebih rumit pada multicore

Multi-Processor Scheduling

Multi-Processor Scheduling adalah mekanisme penjadwalan CPU pada sistem dengan **lebih dari satu prosesor**. Tujuannya adalah mendistribusikan tugas secara efisien di antara prosesor yang tersedia.

↔ Asymmetric vs Symmetric

- 1 **Asymmetric Multiprocessing (AMP):** Satu prosesor bertindak sebagai master, lainnya sebagai slave
- 2 **Symmetric Multiprocessing (SMP):** Setiap prosesor dapat menjadwalkan tugas secara independen
- i **SMP lebih umum digunakan pada sistem modern**

📍 Processor Affinity

- 1 **Soft Affinity:** OS mencoba menjaga proses pada CPU yang sama
- 2 **Hard Affinity:** Proses dibatasi untuk berjalan hanya pada CPU tertentu
- i **Manfaat:** Data tetap di cache CPU, mengurangi cache miss

⚖ Load Balancing

- 1 **Push Migration:** Prosesor yang sibuk memindahkan tugas ke prosesor yang lebih longgar
- 2 **Pull Migration:** Prosesor yang menganggur menarik tugas dari prosesor yang sibuk
- i **Tujuan:** Menghindari prosesor yang kelebihan beban atau menganggur

🏠 Multicore Processors

- 1 **Memory Stall:** Proses menunggu data yang tidak ada di cache
- 2 **Coarse-Grained Multithreading:** Switch thread saat terjadi latency tinggi
- 3 **Fine-Grained Multithreading:** Switch thread setiap siklus instruksi

📋 Kelebihan dan Kekurangan Multi-Processor Scheduling

👍 Kelebihan

- Throughput tinggi melalui eksekusi paralel
- Utilisasi sumber daya lebih baik
- Skalabilitas untuk aplikasi besar
- Responsivitas sistem lebih tinggi

🗨 Kekurangan

- Implementasi dan pemeliharaan kompleks
- Load balancing sulit dicapai
- Overhead migrasi proses
- Sinkronisasi antar prosesor

Real-Time CPU Scheduling

Real-Time CPU Scheduling adalah penjadwalan CPU untuk sistem yang memerlukan **respons tepat waktu**. Sistem ini digunakan pada aplikasi kritis seperti kontrol industri, medis, dan otomotif.

Komponen Real-Time Systems

-  **Real-Time Tasks:** Tugas dengan batasan waktu ketat
-  **Priority-Based Scheduling:** Tugas prioritas tinggi dieksekusi pertama
-  **Deadlines & Timing Constraints:** Batas waktu yang harus dipenuhi
-  **Interrupt Handling:** Mekanisme untuk menangani event eksternal

Klasifikasi Real-Time Systems

-  **Hard Real-Time Systems:** Deadline harus dipenuhi, jika tidak akan menyebabkan kerusakan kritis
-  **Soft Real-Time Systems:** Deadline dapat terlewat tanpa konsekuensi serius

 Pacemaker

 Kontrol Penerbangan

 Streaming Video

 Video Conference

Algoritma Penjadwalan

-  **Rate Monotonic Scheduling (RMS):** Prioritas berdasarkan periode tugas
-  **Earliest Deadline First (EDF):** Tugas dengan deadline terdekat dieksekusi pertama
-  **Deadline Monotonic (DM):** Prioritas berdasarkan deadline relatif
-  **Predictable Round Robin (PRR):** Variasi Round Robin untuk sistem real-time

Langkah-Langkah Penjadwalan

-  **Identifikasi Tugas:** Tentukan tugas real-time yang perlu dijadwalkan
-  **Pemilihan Algoritma:** Pilih algoritma yang sesuai dengan karakteristik tugas
-  **Penetapan Prioritas:** Tetapkan prioritas berdasarkan kepentingan dan deadline
-  **Analisis Waktu:** Verifikasi semua tugas memenuhi deadline
-  **Implementasi & Pengujian:** Implementasikan dan uji performa sistem

Perbandingan Algoritma Real-Time

Kelebihan

- Respons tepat waktu untuk tugas kritis
- Prioritas jelas untuk tugas penting
- Prediktabilitas tinggi
- Dapat dioptimalkan untuk aplikasi spesifik

Tantangan

- Kompleksitas implementasi tinggi
- Memerlukan perencanaan sumber daya cermat
- Sulit menangani beban berlebih
- Memerlukan pengetahuan domain aplikasi

Operating System Examples: Linux

Linux menggunakan **penjadwalan preemptif berbasis prioritas** yang dirancang untuk memberikan keseimbangan antara responsivitas dan throughput. Kernel Linux memiliki arsitektur penjadwalan yang modular dan fleksibel.

Komponen Utama Linux Scheduler

-  **Runqueue:** Struktur data yang menyimpan proses siap untuk dieksekusi pada setiap CPU
-  **Scheduling Classes:** Modul yang mengimplementasikan berbagai algoritma penjadwalan
-  **Scheduler Core:** Komponen utama yang mengkoordinasikan kelas penjadwalan

Kelas Penjadwalan Linux

-  Linux menggunakan **arsitektur berbasis kelas** yang memungkinkan berbagai algoritma penjadwalan berjalan bersama

 **DL (Deadline)**

 **RT (Realtime)**

 **FAIR (CFS)**

 **IDLE**

- DL:** Tugas dengan deadline ketat (audio/video processing)
- RT:** Tugas sensitif latensi (IRQ threads)
- FAIR (CFS):** Sebagian besar tugas di OS (algoritma default)

Fungsi Utama Scheduler

 **schedule()** API publik untuk memanggil scheduler

 **__schedule()** Fungsi internal yang memilih proses berikutnya

 **scheduler_tick()** Dipanggil secara periodik untuk update prioritas

 **pick_next_task()** Memilih proses berikutnya dari kelas penjadwalan

CFS (Completely Fair Scheduler)

- ★ **Algoritma default** untuk proses normal di Linux
-  Bertujuan memberikan **bagian CPU yang adil** untuk semua proses
-  Menggunakan **virtual runtime** untuk menentukan prioritas
-  Proses dengan virtual runtime terendah akan dijalankan berikutnya
-  Secara otomatis menyesuaikan prioritas berdasarkan perilaku proses

Operating System Examples: Windows

Windows menggunakan **penjadwalan preemptive berbasis prioritas** yang dirancang untuk memberikan responsivitas tinggi untuk aplikasi interaktif sambil memastikan proses background tetap berjalan.

Dispatcher Windows

- ! Dispatcher menggunakan **skema prioritas 32-level** untuk menentukan urutan eksekusi thread
- ↔ Dispatcher menggunakan antrian untuk setiap tingkat prioritas
- 🔍 Dispatcher memeriksa antrian dari prioritas tertinggi ke terendah
- 🕒 Thread dengan prioritas tertinggi yang siap akan dieksekusi

Kelas Prioritas Windows

1 Variable Class (Prioritas 1-15)

1 2 ... 15

Untuk aplikasi normal, prioritas dapat berubah

2 Real-Time Class (Prioritas 16-31)

16 17 ... 31

Untuk aplikasi real-time, prioritas tetap

⚙️ Memory Management (Prioritas 0)

0

Thread khusus untuk manajemen memori sistem

Mekanisme Penjadwalan

- 1 **Preemptive:** Thread dengan prioritas lebih tinggi dapat menginterupsi thread yang sedang berjalan
- 2 **Quantum Time:** Setiap thread mendapat jatah waktu CPU, setelah itu akan dipreempt
- 3 **Dynamic Priority:** Prioritas thread dapat berubah berdasarkan perilaku (interaktif vs CPU-bound)
- 4 **Aging:** Thread yang menunggu lama akan mendapat peningkatan prioritas

Task Scheduler Windows 10

- i Tool untuk **otomatisasi tugas** yang berjalan pada waktu tertentu atau saat event tertentu terjadi

🕒 Maintenance

🕒 Alarm

🔄 Updates

📁 Backup

🛡️ Security Scan

- 🔋 **Battery Saver** memodifikasi Task Scheduler untuk menghemat energi
- 🔌 Tugas dapat diatur untuk berjalan saat sistem idle atau saat pengguna login

Operating System Examples: Solaris

Solaris menggunakan **Fair Share Scheduler (FSS)** yang mengalokasikan sumber daya CPU berdasarkan "shares" yang ditetapkan untuk setiap proyek, memberikan kontrol yang lebih baik atas alokasi sumber daya.

Konsep "Share" di Solaris

- Share** adalah bagian dari sumber daya CPU yang dialokasikan ke sebuah proyek
- Share bukan persentase, melainkan **kepentingan relatif** dibanding proyek lain
- Proses dengan **no shares** selalu berjalan pada prioritas terendah (0)

Contoh Alokasi Share

Jika Proyek A memiliki 1 share dan Proyek B memiliki 3 shares, maka Proyek A mendapat 25% dan Proyek B mendapat 75% dari sumber daya CPU saat keduanya aktif.

Status Proyek

- Idle**: Tidak ada proses dalam proyek yang menggunakan sumber daya CPU
- Aktif**: Setidaknya satu proses dalam proyek menggunakan sumber daya CPU
- Ketika lebih banyak proyek menjadi aktif, alokasi CPU untuk setiap proyek berkurang
- Proyek yang berjalan sendiri akan mendapat **100% CPU** terlepas dari jumlah share

Model Prioritas Solaris

Solaris mengenali **170 prioritas berbeda** (0-169)

 TS (0-59)

 IA (0-59)

 FSS (0-59)

 FX (0-59)

 SYS (60-99)

 RT (100-159)

- TS (Timesharing)**: Prioritas disesuaikan berdasarkan utilitas CPU
- RT (Real-time)**: Prioritas tetap dengan quantum waktu tetap

Konfigurasi FSS

- Dikelola oleh **name service** sebagai properti database proyek
- Shares ditentukan melalui **project.cpu-shares** di database proyek
- Perubahan shares dapat dilakukan dengan perintah **prctl**

```
# Mengubah shares untuk proyek x-files
prctl -r -n project.cpu-shares -v 3 -i project x-files
```

- Maksimum shares per proyek: **65535**

Algorithm Evaluation

Evaluasi algoritma penjadwalan CPU penting untuk **menentukan performa** dan **kecocokan** algoritma untuk situasi tertentu. Ada beberapa metode evaluasi yang dapat digunakan.

Deterministic Modeling

-  Menggunakan **beban kerja yang telah ditentukan** untuk mengevaluasi algoritma
-  Menghitung metrik performa secara matematis

Kelebihan

- ✓ Tepat dan cepat
- ✓ Mudah dihitung

Kekurangan

- ✗ Hanya berlaku untuk beban kerja spesifik
- ✗ Tidak representatif untuk situasi nyata

Queuing Models

-  Menggunakan **teori antrian** untuk menganalisis performa
-  Menggunakan distribusi probabilitas untuk waktu burst dan kedatangan

$$n = \lambda \times w$$

-  n : panjang antrian rata-rata
 λ : tingkat kedatangan rata-rata
 w : waktu tunggu rata-rata

Simulations

-  **Simulasi komputer** untuk meniru perilaku sistem
-  Variabel jam ditingkatkan secara bertahap
-  Statistik dikumpulkan pada setiap tick jam

Kelebihan

- ✓ Dapat menggunakan data nyata (trace)
- ✓ Hasil lebih akurat

Kekurangan

- ✗ Waktu eksekusi lama
- ✗ Implementasi kompleks

Implementation

-  **Implementasi algoritma** pada sistem nyata
-  Mengukur performa dengan beban kerja nyata

Kelebihan

- ✓ Hasil paling akurat
- ✓ Mencakup semua faktor nyata

Kekurangan

- ✗ Biaya implementasi tinggi
- ✗ Memerlukan lingkungan produksi
- ✗ Dapat mengganggu pengguna

Summary

CPU Scheduling adalah **komponen krusial** dalam sistem operasi yang menentukan efisiensi dan responsivitas sistem. Mari kita ringkas poin-poin penting dari setiap topik yang telah dibahas.

Basic Concepts

- ✓ Proses memutuskan tugas mana yang menggunakan CPU
- ✓ Terminologi: Arrival Time, Burst Time, Turnaround Time
- ✓ CPU-I/O Burst Cycle: siklus eksekusi proses

Scheduling Criteria

- ✓ CPU Utilization: memaksimalkan penggunaan CPU
- ✓ Throughput: jumlah proses yang selesai per waktu
- ✓ Turnaround Time, Waiting Time, Response Time

Scheduling Algorithms

- ✓ Non-Preemptive: FCFS, SJF, LJF
- ✓ Preemptive: Priority, Round Robin, SRTF
- ✓ Multilevel: Queue dan Feedback Queue

Thread Scheduling

- ✓ User-Level Threads (ULT) vs Kernel-Level Threads (KLT)
- ✓ Lightweight Process (LWP) sebagai antarmuka
- ✓ Contention Scope: PCS vs SCS

Multi-Processor Scheduling

- ✓ AMP vs SMP: Asymmetric vs Symmetric
- ✓ Processor Affinity: Soft vs Hard
- ✓ Load Balancing: Push vs Pull Migration

Real-Time CPU Scheduling

- ✓ Hard vs Soft Real-Time Systems
- ✓ Algoritma: RMS, EDF, DM, PRR
- ✓ Deadline dan Timing Constraints

Operating System Examples

- ✓ Linux: CFS, RT, DL, IDLE scheduling classes
- ✓ Windows: 32-level priority scheme
- ✓ Solaris: Fair Share Scheduler (FSS)

Algorithm Evaluation

- ✓ Deterministic Modeling: beban kerja tetap
- ✓ Queuing Models: Little's Formula ($n=\lambda w$)
- ✓ Simulations vs Implementation

Kesimpulan

- ✓ Tidak ada algoritma "terbaik" untuk semua situasi
- ✓ Pemilihan algoritma tergantung kebutuhan sistem
- ✓ CPU Scheduling terus berkembang dengan hardware

Latihan Soal

Berikut adalah beberapa soal latihan untuk menguji pemahaman Anda tentang **CPU Scheduling**. Cobalah jawab soal-soal ini untuk memperkuat konsep yang telah dipelajari.

- 1** Diberikan tiga proses dengan waktu burst sebagai berikut: P1 = 24, P2 = 3, P3 = 3. Jika semua proses tiba pada waktu 0, hitunglah Average Waiting Time untuk algoritma FCFS dan SJF. ★ Mudah

💡 Petunjuk: Buat Gantt Chart untuk setiap algoritma, hitung Waiting Time untuk setiap proses, lalu rata-ratakan.

- 2** Sebuah sistem menggunakan algoritma Round Robin dengan time quantum = 4. Diberikan proses: P1 (waktu burst = 20), P2 (waktu burst = 4), P3 (waktu burst = 2). Jika semua proses tiba pada waktu 0, berapa Average Turnaround Time? ★ Sedang

💡 Petunjuk: Perhatikan bagaimana proses berpindah antara CPU dan antrian setiap time quantum.

- 3** Sistem real-time memiliki tiga tugas: T1 (periode = 30, waktu eksekusi = 10), T2 (periode = 50, waktu eksekusi = 25), T3 (periode = 100, waktu eksekusi = 20). Apakah sistem ini dapat dijadwalkan menggunakan Rate Monotonic Scheduling? Jelaskan alasannya. ★ Sulit

💡 Petunjuk: Gunakan uji kelayakan CPU Utilization: $U = \sum (C_i/T_i) \leq n(2^{1/n} - 1)$, di mana n adalah jumlah tugas.

- 4** Bandingkan kelebihan dan kekurangan antara algoritma SJF dan Round Robin dalam konteks sistem interaktif vs sistem batch. Algoritma mana yang lebih cocok untuk setiap jenis sistem dan mengapa? ★ Sedang

💡 Petunjuk: Pertimbangkan faktor-faktor seperti responsivitas, fairness, dan throughput.

Diskusikan jawaban Anda dengan teman sekelas atau dosen pengampu

Terima Kasih

CPU Scheduling - Sistem Operasi

Program Studi Teknik Informatika, Universitas Muhammadiyah Jember, 2025

